

Swarm Decision-Making via Evolutionary Optimization in Pursue-Containment Scenarios

1st Xiaoyue JinDepartment of Data Science and
Artificial IntelligenceThe Hong Kong Polytechnic University
Hong Kong, China
xiaoyue.jin@connect.polyu.hk2nd Yinglan FengDepartment of Data Science and
Artificial IntelligenceThe Hong Kong Polytechnic University
Hong Kong, China
yinglfeng@polyu.edu.hk3rd Ran ChengDepartment of Data Science and
Artificial IntelligenceThe Hong Kong Polytechnic University
Hong Kong, China
ranchengcn@gmail.com4th Dengxiu YuDepartment of Artificial Intelligence, Optics and Electronics (iOPEN) Department of Data Science and Artificial Intelligence
Northwestern Polytechnical University
Xi'an, China
yudengxiu@nwpu.edu.cn5th Kay Chen TanThe Hong Kong Polytechnic University
Hong Kong, China
kctan@polyu.edu.hk

Abstract—Swarm systems require effective decision-making for timely action coordination. The pursuit-containment task, in particular, highlights the necessity of both intra-swarm cooperation and external competition. However, existing learning-based methods often suffer from poor coordination and are typically limited to small-scale swarms. To address these issues, this paper proposes an evolutionary optimization framework that efficiently facilitates real-time decision making for large-scale swarms, enabling the formation of a uniformly distributed containment circle around targets.

Index Terms—Evolutionary Optimization, Swarm System, Pursue-Containment.

I. INTRODUCTION

Swarm systems, consisting of simple agents like unmanned aerial vehicles or ground robots, have gained significant attention due to their cost-effectiveness and robust coverage capabilities [1]. These systems coordinate to perform complex tasks by interacting with the environment, exchanging information, and making task-driven decisions [2]. Real-time decision-making involves generating sequences of decisions tailored to specific tasks. The pursuit-containment task, a challenging example of swarm confrontation, requires both cooperation and competition. Agents must move from random initial positions to form a collision-free, uniformly distributed containment circle around the target to prevent escape [3] with complex constraints and irregular feasible regions.

However, widely adopted learning-based methods like Multi-Agent Deep Deterministic Policy Gradient (MADDPG) often suffer from limited intra-swarm coordination due to their distributed training architectures [4], which can result in suboptimal behaviors where some agents fail to accomplish

the task effectively. In addition, high computational cost and training times [5] significantly make it difficult to apply such methods to swarms with a large number of agents [6]. Thus, this paper proposes an evolutionary optimization framework for Pursue-Containment (EOPC) that enables fast and efficient computation of coordinated decisions, while also scaling well with the number of agents involved in the swarm.

II. EVOLUTIONARY OPTIMIZATION FOR PURSUE-CONTAINMENT

The swarm system consists of N agents, where the state of agent i at time t is denoted by $\mathbf{S}_i(t) = (x_i^t, y_i^t)$, representing its coordinates along the x - and y -axes in the global coordinate frame. The state is updated based on the decision u :

$$\begin{cases} x_i^{t+1} = x_i^t + V_{i,x}, \\ y_i^{t+1} = y_i^t + V_{i,y}, \end{cases} \quad (1)$$

where $V_{i,x}$ and $V_{i,y}$ are the x - and y -axis components of the velocity vector V_i , which serve as the decision u_i for the system. Specifically, the decision can be written as $u_i(t) = (V_{i,x}, V_{i,y})$. Therefore, computing an effective decision u that enables the swarm to efficiently complete the task constitutes the primary objective of the decision-making algorithm.

A. Problem Decomposition

Due to the uncertainty of the target's trajectory, decisions must be generated repeatedly in real time. Thus, the pursuit-containment problem is divided into discrete time slices with interval δ_t . At each time t , agents' states are treated as known parameters in a static subproblem, and the optimal decision u^* is computed using the proposed optimization framework to minimize the objective function $\mathcal{J}$:

$$\mathbf{u}^*(t) = \arg \min_{\mathbf{u}} \mathcal{J}(\mathbf{S}(t), \mathbf{u}), \quad (2)$$

where $\mathbf{S}(t)$ represents the current states of all agents.

This work was supported in part by National Natural Science Foundation of China (Grant No. U21A20512), Research Grants Council of the Hong Kong SAR (Grant No. C5052-23G, PolyU15229824, PolyU15218622, PolyU15215623), and The Hong Kong Polytechnic University (Project IDs: P0053758, P0051130, P0052694).

With the decision $\mathbf{u}^*(t)$, the states of all agents are updated, resulting in their new states at the next time slice $t + \delta_t$:

$$\mathbf{S}(t + \delta_t) = g(\mathbf{S}(t), \mathbf{u}^*(t)), \quad (3)$$

where $g(\cdot)$ is the state update function of agents as shown in (1). This process is repeated until the task is accomplished.

B. Static Subproblem Modeling

To obtain the optimal decision $\mathbf{u}^*$, directly optimizing next-step Cartesian coordinates (x^{t+1}, y^{t+1}) for N agents results in a high-dimensional $M \times (2N)$ search space, complicating variable decoupling. To address this, we transform agent positions into polar coordinates (r_i, θ_i) centered at the target, with θ_i increasing counterclockwise. The coordinates of the containment circle are fixed in radius $\bar{r}$ and variable angle θ_i .

Then, the angular components θ_i of all agents are collected to form the following optimization variable matrix:

$$\Theta = \begin{bmatrix} \theta_{1,1} & \theta_{1,2} & \cdots & \theta_{1,N} \\ \theta_{2,1} & \theta_{2,2} & \cdots & \theta_{2,N} \\ \vdots & \vdots & \ddots & \vdots \\ \theta_{M,1} & \theta_{M,2} & \cdots & \theta_{M,N} \end{bmatrix}, \quad (4)$$

where M is the size of population. During the optimization process, two objectives are considered in this paper:

$$\begin{aligned} F_1(t) &= \min \frac{1}{N} \sum_{i=1}^N \sqrt{(x_i^t - x_i^{\theta_i, t})^2 + (y_i^t - y_i^{\theta_i, t})^2}, \\ F_2(t) &= \min \max_i (\theta^i - \theta^{i-1}), \end{aligned} \quad (5)$$

s.t.

$$\begin{aligned} \theta_i &\neq \theta_j, \quad \forall i \neq j, \\ (x_i^{t+1}, y_i^{t+1}) &\neq (x_j^{t+1}, y_j^{t+1}), \quad \forall i \neq j, \\ (x_i^{t+1}, y_i^{t+1}) &\neq (x_0^{t+1}, y_0^{t+1}), \quad \forall i, \end{aligned}$$

where (x_i^t, y_i^t) is the current position of agent i at time t , and $(x_i^{\theta_i, t}, y_i^{\theta_i, t})$ is its assigned target position on the containment circle, derived from polar coordinates $(\bar{r}, \theta_i)$. Objective F_1 minimizes the average displacement between agents and their targets, while F_2 promotes uniform angular spacing. Let $[\theta^1, \dots, \theta^N]$ be the sorted version of $[\theta_1, \dots, \theta_N]$; a smaller F_2 implies better formation uniformity.

The constraints ensure collision-free assignments by preventing duplicate angles, agent overlap, and collisions with the target. The angular matrix Θ is optimized using SMPSO [1] to obtain Θ^* , and the decisions are computed as $\mathbf{u}^* = (V \cos \theta^*, V \sin \theta^*)$, with V denoting constant speed.

III. SIMULATION RESULTS

In the simulation, the target was moved randomly at a velocity of 0.8, with agents starting from random positions. The containment radius was set at $\bar{r} = 0.5$ and the pursuit speed at $V = 1$. Each optimization ran for 50 generations. Fig. 1 shows the convergence of F_1 and F_2 at $t = 4$, confirming the effectiveness of the framework in solving subproblems.

Additionally, EOPC is compared with MADDPG by evaluating average subproblem solving time (over 100 runs) and

reward convergence time as the number of agents increases. The results show that EOPC scales more efficiently, confirming its suitability for large-scale swarm systems.

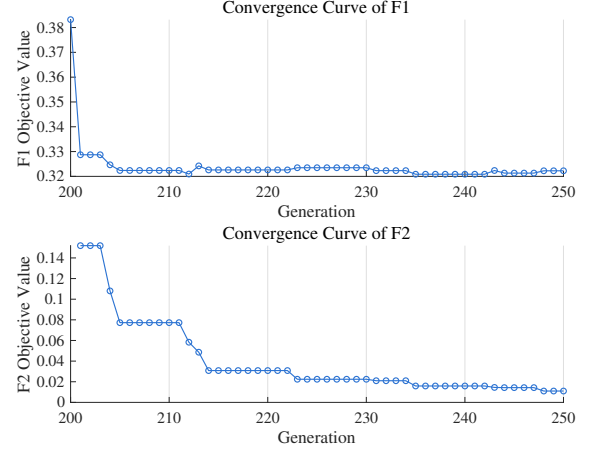


Fig. 1. Convergence curves of objective functions F_1 and F_2 during the optimization of a static subproblem.

TABLE I
COMPARISON OF EOPC AND MADDPG

N	Avg-T (EOPC)	Con-T (MADDPG)	IR (EOPC)	IR (MADDPG)
5	0.2112	4200	0.00%	0%
8	0.3201	7000	52.00%	67.00%
10	0.3937	10200	86.00%	143.00%
12	0.4734	12900	124.00%	207.00%
14	0.5487	27600	159.00%	557.00%

* Avg-T: average solving time; Con-T: convergence time; IR: increase ratio.

IV. CONCLUSION

This paper presented the EOPC in swarm systems. By decomposing the task and applying polar transformation, it reduces dimensionality and improves efficiency. The method jointly optimizes pursuit distance and agent distribution with collision avoidance. Using SMPSO, it enables real-time, coordinated containment and scales well to large swarms, making it suitable for embodied evolutionary robotics.

REFERENCES

- [1] Y. Feng, L. Feng, Y. Hou, and K. C. Tan, "Large-scale optimization via evolutionary multitasking assisted random embedding," in Proc. IEEE Congr. Evol. Comput. (CEC), 2020, pp. 1–8.
- [2] L. Wang, T. Qiu, Z. Pu, and J. Yi, "A cooperation and decision-making framework in dynamic confrontation for multi-agent systems," Comput. Electr. Eng., vol. 118, p. 109300, 2024.
- [3] Z. Zhou and H. Xu, "Decentralized optimal large-scale multi-player pursuit-evasion strategies: A mean field game approach with reinforcement learning," Neurocomputing, vol. 484, pp. 46–58, 2022.
- [4] R. Lowe, Y. I. Wu, A. Tamar, J. Harb, O. P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," Adv. Neural Inf. Process. Syst., 2017.
- [5] B. Li, J. Wang, C. Song, Z. Yang, K. Wan, and Q. Zhang, "Multi-UAV roundup strategy method based on deep reinforcement learning CEL-MADDPG algorithm," Expert Syst. Appl., vol. 245, p. 123018, 2024.
- [6] H. Liang, Z. Chen, and K. Jing, "Actor-critic reinforcement learning algorithms for mean field games in continuous time, state and action spaces," Appl. Math. Optim., vol. 89, no. 3, p. 72, 2024.